Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Revision of Perceptron Algorithm

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Problem Formulation

Define δ_i as:

If X ∈ ***class I*** and *misclassified as* ***Class II*** then *d(X)* will be **<0** ⇒ $\delta_i = -1$ ⇒ $d(X)\delta_i > 0$

If X ∈ ***class II*** and *misclassified as* ***Class I*** then *d(X)* will be **>0** ⇒ $\delta_i = +1$ ⇒ $d(X)\delta_i > 0$

Redefine Error function as:

$$\text{Error} = \sum_{\text{all misclassified vectors } (i)} \delta_i W^T X_i$$

⇒ *($W^T X_i$) takes positive and negative values and δ_i has the same sign*

*Summation is always **positive***

*Minimize the Error using **Gradient Descent** with step **η***

$$W^{(\text{updated})} = W^{(\text{old})} - \eta \frac{\partial \text{Error}}{\partial W} \quad \textbf{where} \quad \frac{\partial \text{Error}}{\partial W} = \sum_{\text{misclassified vectors}} \delta_i X_i$$

$$W^{(\text{updated})} = W^{(\text{old})} - \eta \sum_{\text{misclassified vectors ONLY } (i)} \delta_i X_i$$

Update after each sample

$$W^{(\text{updated})} = W^{(\text{old})} - \eta \delta_i X_i$$

Visualization of Single Perceptron

$$X = \begin{bmatrix} x_1; \\ x_2; \\ \dots \\ x_n \end{bmatrix} \rightarrow X = \begin{bmatrix} 1; \\ x_1; \\ x_2; \\ \dots \\ x_n \end{bmatrix}$$

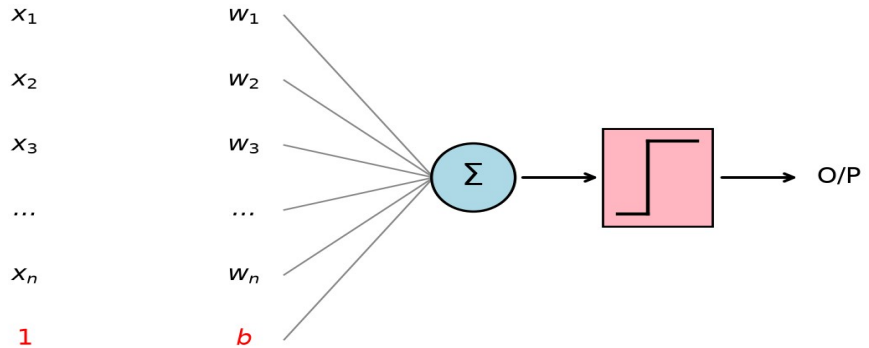
Augmented Vector

$$W = \begin{bmatrix} b; \\ w_1; \\ w_2; \\ \dots \\ w_n \end{bmatrix}$$

Model Weights

$$d(X) = W^T X = w_1x_1 + w_2x_2 + \dots\dots + w_nx_n + b*1$$

}
>0 ⇒ Class I
<0 ⇒ Class II
=0 ⇒ Undefined Class

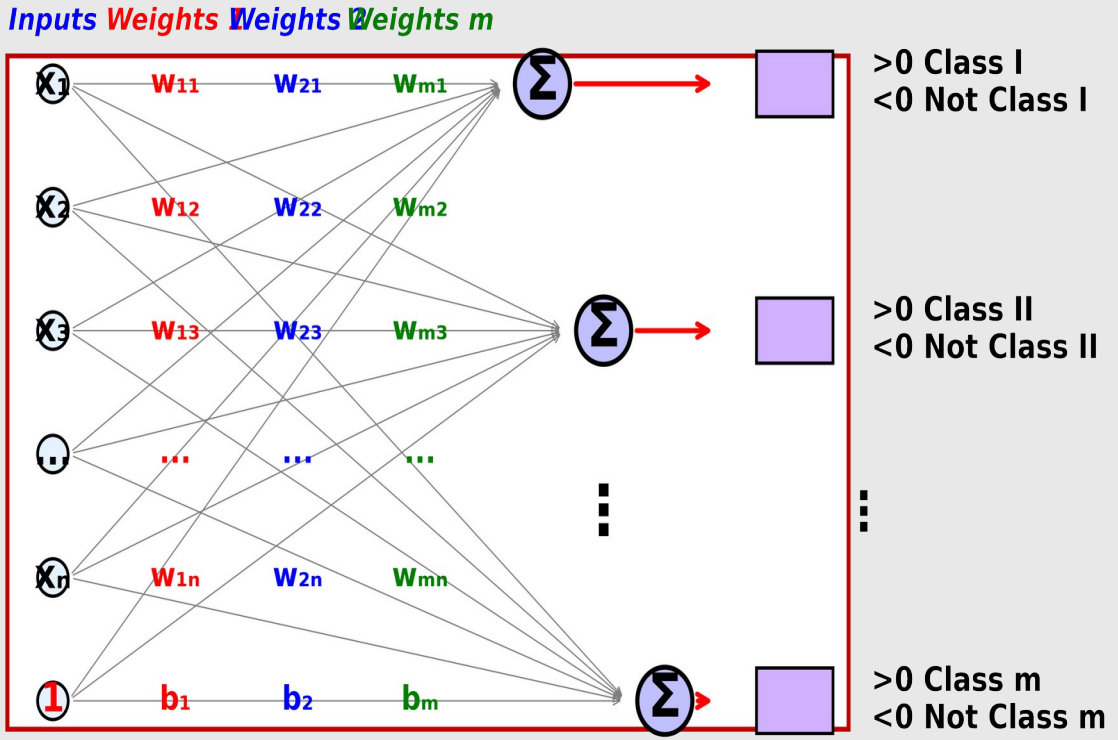


Binary Classification (2 Classes)

Visualization of Single Layer Perceptron

$$d_1(X) = W_1^T X = w_{11}x_1 + w_{12}x_2 + \ldots + w_{1n}x_n + b_1 \cdot 1$$
$$d_2(X) = W_2^T X = w_{21}x_1 + w_{22}x_2 + \ldots + w_{2n}x_n + b_2 \cdot 1$$
$$d_m(X) = W_m^T X = w_{m1}x_1 + w_{m2}x_2 + \ldots + w_{mn}x_n + b_m \cdot 1$$

Multi-Class Classification



Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Adding non-Linearity to Perceptrons

(Activation Functions)

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

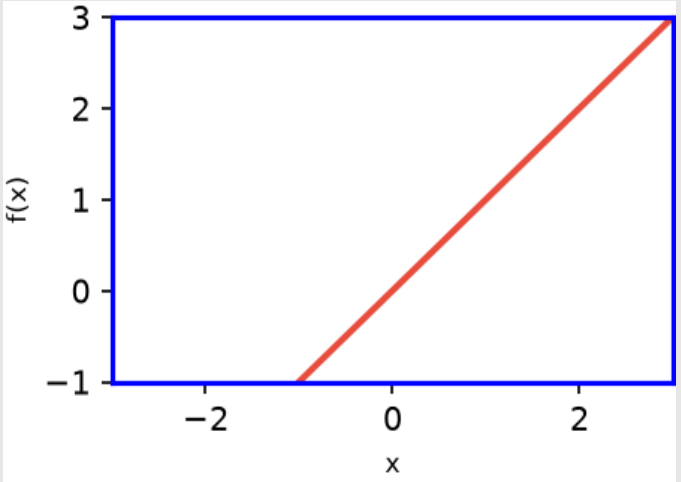
Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

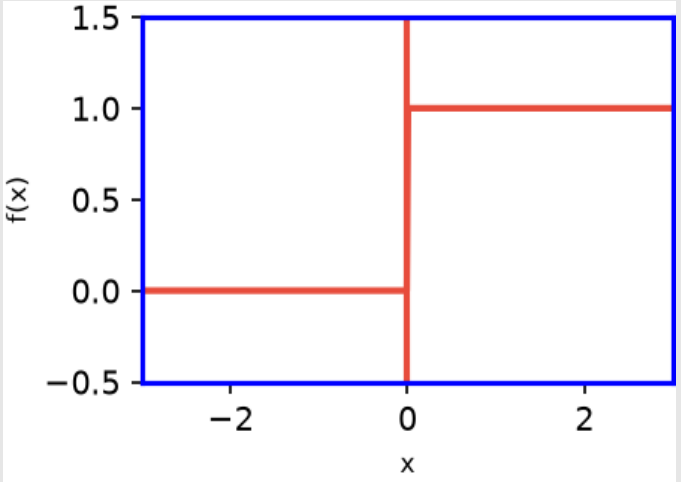
Test Single Perceptron with [AND – OR – XOR] Logic (Binary Classification)

Linear and Unit Step Activation Functions

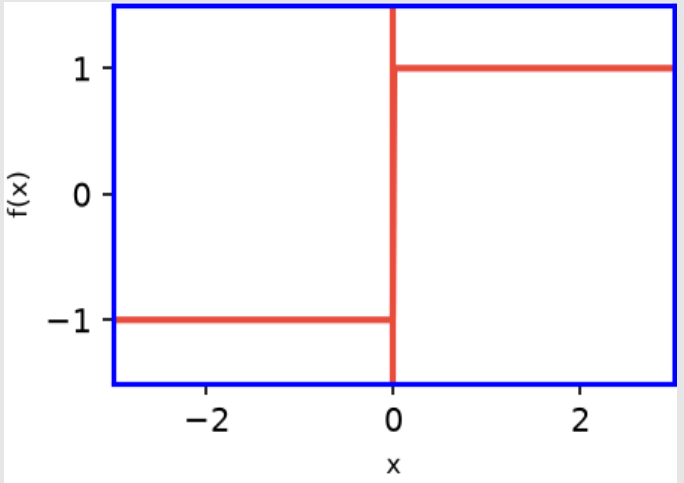


F(x) = x

Linear (No Activation)



F(x) = Uni-polar Unit Step = { 0 for x<0 ; 1 for x ≥ 0 }



F(x) = Bi-polar Unit Step = { -1 for x<0 ; 1 for x ≥ 0 }

Sigmoid vs Tanh

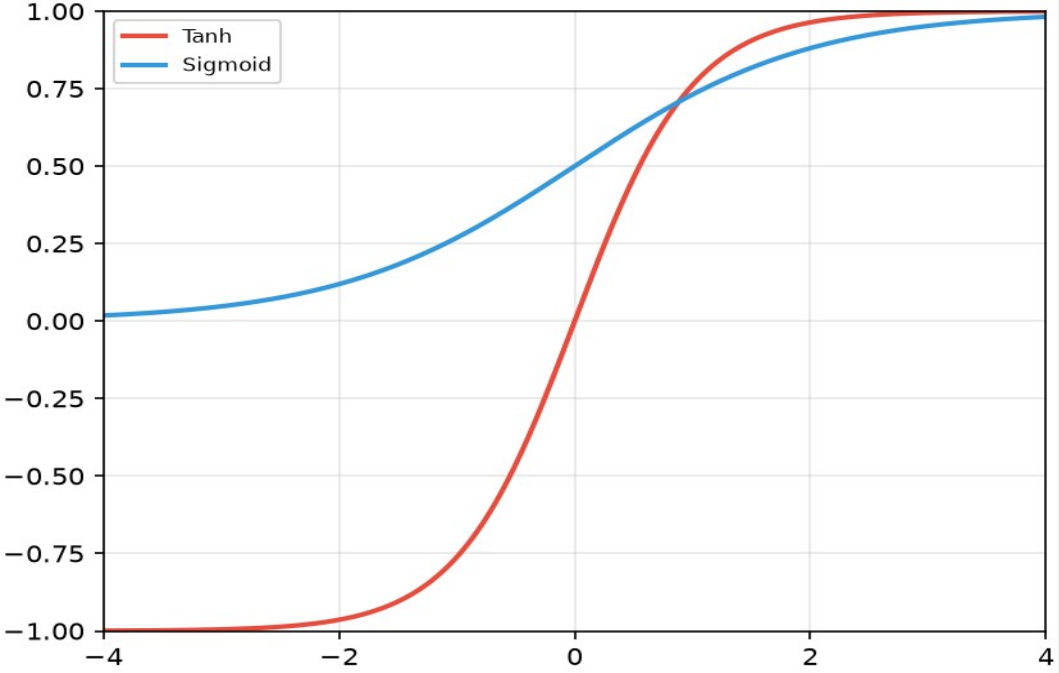
f(x) = Sigmoid(x) = 1/(1+e^-x) ⇒ f'(x) = f(x)(1-f(x))

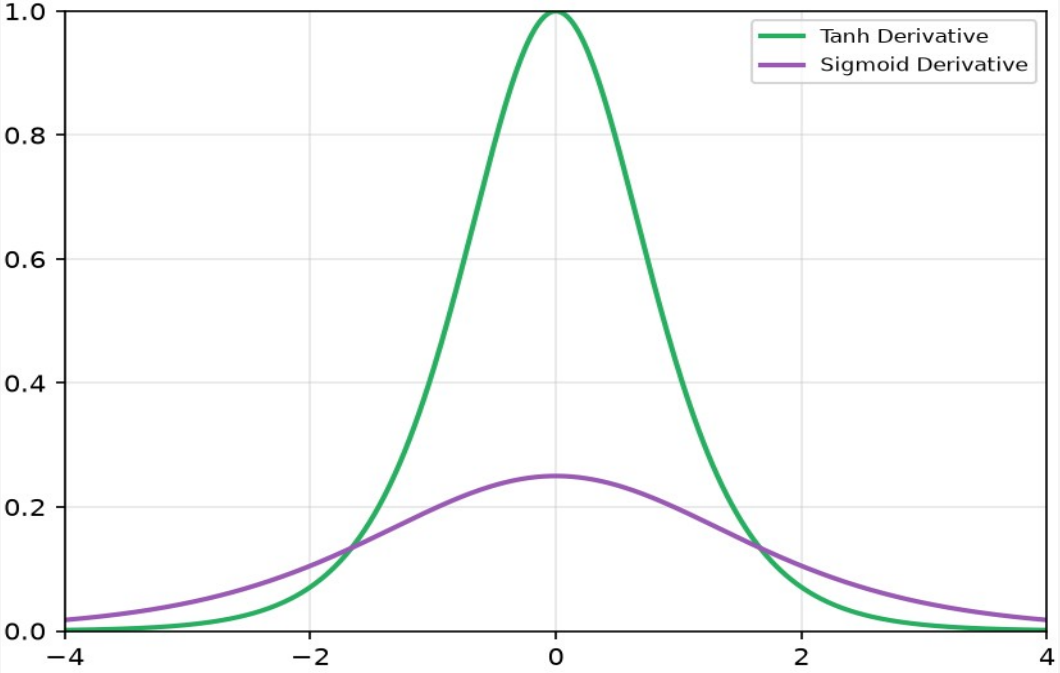
at x=0 f(0)=0.5 f'(0)=0.25

f(x) = tanh(x) = sinh x / cosh x ⇒ f'(x) = 1-f(x)^2

at x=0 f(0)=0 f'(0)=1

Sigmoid vs Tanh



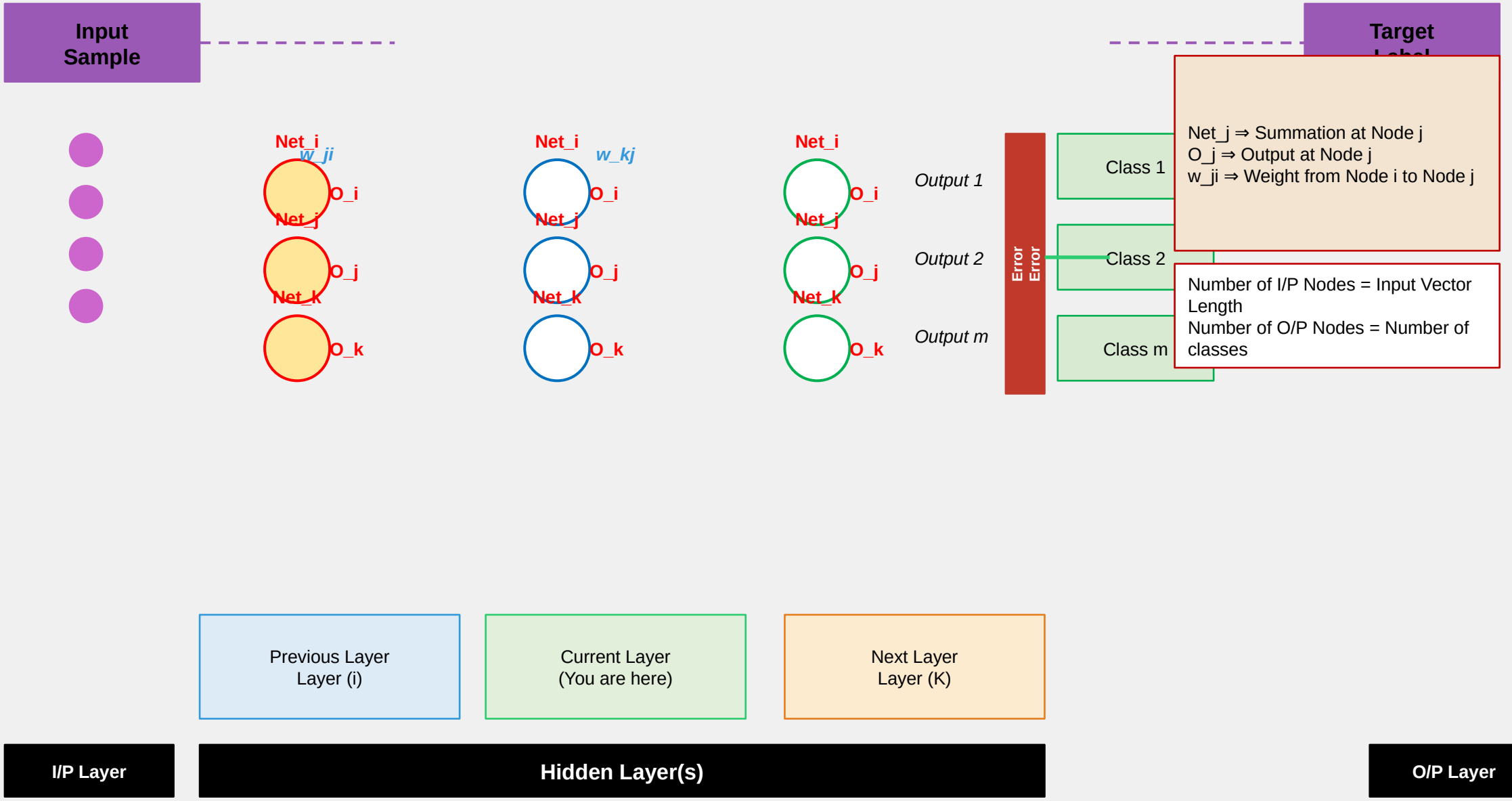


[1] The range of the sigmoid function is fixed at 0 to 1, whereas the range of the tanh function is fixed at -1 to 1.
[2] The sigmoid function has a slower rate of change than the tanh function, which means that the derivative of the sigmoid function will be smaller than the derivative of the tanh function for a given input

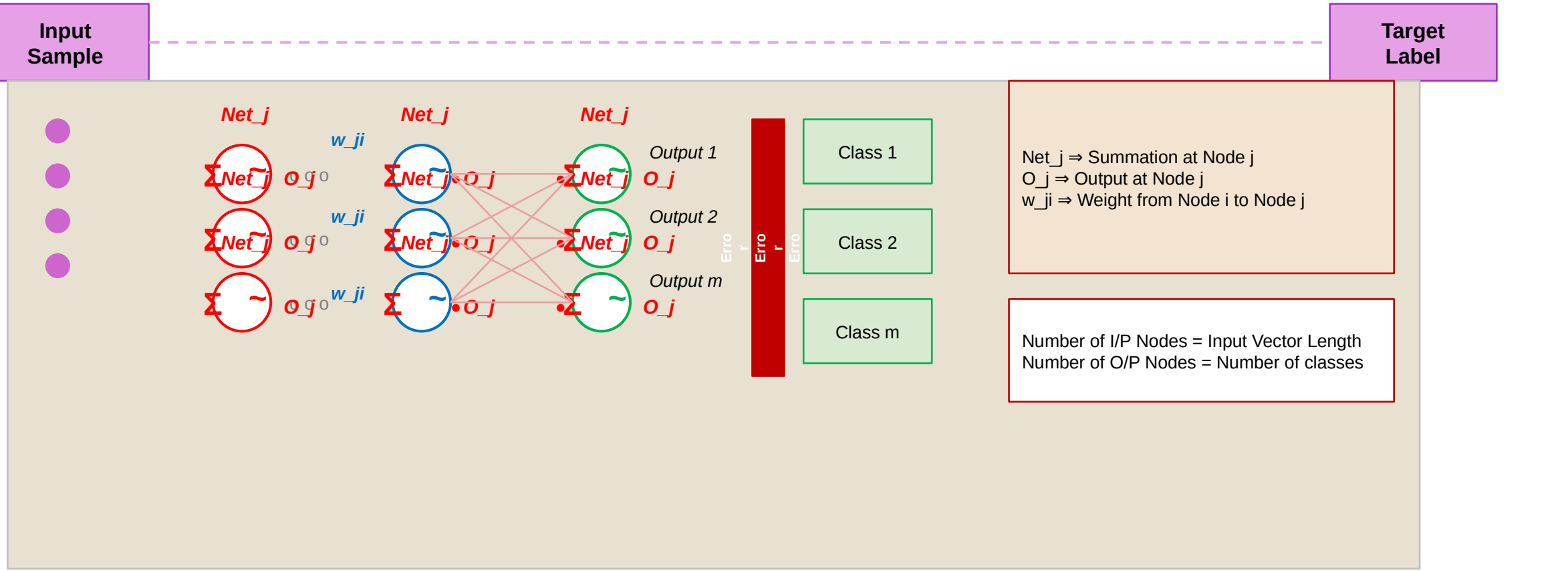
Multi-Layer Neural Network

The complete Picture

Neural Network Model



Neural Network Model



For Layer "j"

$$Net_j = \Sigma_j (w_{ji} O_i)$$

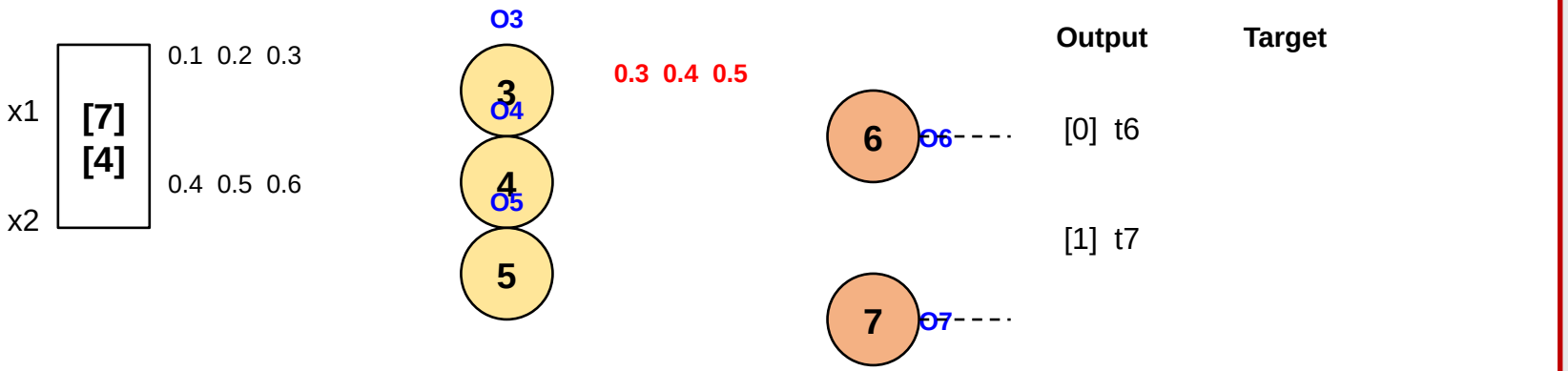
$$O_j = f(Net_j) = 1 / (1 + e^{-Net_j})$$

If Layer "j" is Output Layer:

$$E = \tfrac{1}{2} \Sigma_j (t_j - O_j)^2$$

Example

Bad Weights



Forward

Net3

Net3 = 7 * 0.1 + 4 * 0.4 = 2.3
Net4 = 7 * 0.2 + 4 * 0.5 = 3.4
Net5 = 7 * 0.3 + 4 * 0.6 = 4.5

O3

O3 = 1 / (1 + e^-2.3) = 0.909
O4 = 1 / (1 + e^-3.4) = 0.968
O5 = 1 / (1 + e^-4.5) = 0.989

Net6

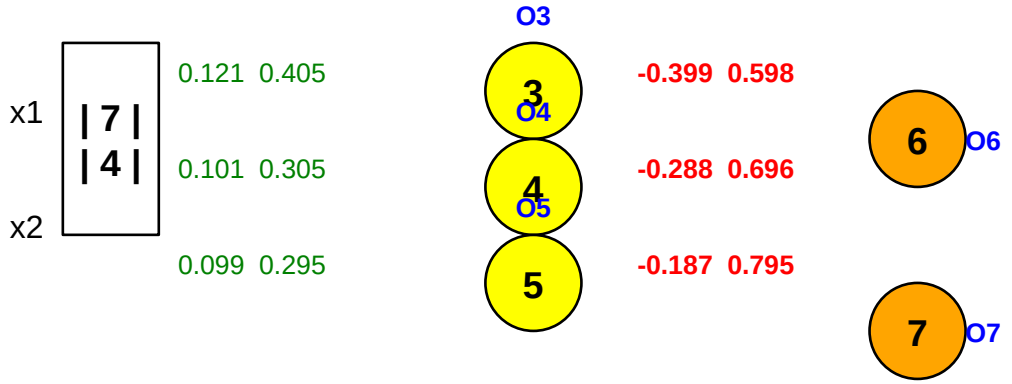
Net6 = 0.909 * 0.3 + 0.968 * 0.4 + 0.989 * 0.5 = 1.1544
Net7 = 0.909 * 0.3 + 0.968 * 0.4 + 0.989 * 0.5 = 1.1544

O6

O6 = 1 / (1 + e^-1.1544) = 0.76
O7 = 1 / (1 + e^-1.1544) = 0.76

Example

Better Weights



Forward

Net3

Net3 = 7 * 0.121 + 4 * 0.405 = 2.146
Net4 = 7 * 0.101 + 4 * 0.305 = 1.626
Net5 = 7 * 0.099 + 4 * 0.295 = 1.574

O3

O3 = 1 / (1 + e^2.146) = 0.895
O4 = 1 / (1 + e^1.626) = 0.836
O5 = 1 / (1 + e^1.574) = 0.828

Net6

Net6 = 0.895 * (-0.399) + 0.836 * (-0.288) + 0.828 * (-0.187) = -0.753
Net7 = 0.895 * 0.598 + 0.836 * 0.696 + 0.828 * 0.795 = 1.775

O6

O6 = 1 / (1 + e^-0.753) = 0.32
O7 = 1 / (1 + e^1.775) = 0.855

How to Update Weights in order to minimize Error ?

Move in the opposite direction of the gradients

Of the whole NN

(Back Propagation Algorithm)